

텔롬엑스 MES 온톨로지 적용 설계

infurs·hisense·naturefood 세 MES가 지금 각자 다르게 부르고 있는 같은 현실을 하나의 객체 모델로 묶고, 그 위에 판단(Logic)·실행(Action)·권한(Security)을 얹는다. 데이터를 옮기는 작업이 아니라, 세 공장의 업무 세계를 같은 언어로 합의하는 작업이다.

문서	작성일	대상 시스템	근거
TX-ONT-DESIGN r1.0	2026-09-19	infurs_db · hisense_db · naturefood	운영 DB 실측 + Palantir Ontology 설계원칙

상태
설계안 — 미착수

목차

§1	지금 왜 — 3사 실측 진단	§2	설계 원칙 7개
§3	공통 코어 온톨로지	§4	사이트별 매핑과 IRI 규칙
§5	Logic — 판단 규칙 등록부	§6	Action — 실행은 MES가 한다
§7	Security — 사람과 AI를 같은 체계로	§8	Decision Lineage
§9	배치 아키텍처와 동기화	§10	단계별 로드맵
§11	금지 목록과 성공 판정	§12	미결 사항

§ 01

지금 왜 — 3사 실측 진단

온톨로지를 "안 해도 되지만 있으면 좋은 것"으로 보면 이 문서는 필요 없다. 이 설계가 필요한 근거는 이미 발생한 장애와 재작업이다. Palantir가 온톨로지 안티패턴으로 열거한 네 가지 — System Silos,

Department Silos, God Object, Kitchen Sink — 가 세 MES에 각각 실물로 존재하며, 그중 하나는 2026-08-07에 실제 회귀 장애를 냈다.

안티패턴	우리 시스템의 실물	이미 치른 비용
System Silos 같은 실체를 시스템마다 따로	infurs에 품목 마스터가 둘. <code>t_item</code> 687행(수주·재고가 참조)과 <code>t_product</code> 661행(사진·도면이 참조)이 서로 FK 없이 독립. 유일한 연결 수단이 <code>t_product.memo</code> 안의 “모델명:” 문자열.	제품사진 366건 중 품목에 붙은 것은 362건, 312 품목은 끝내 미연결. 도면관리 561품목 중 335개만 사진 표시.
Department Silos 부서마다 별도 Customer	infurs 거래처 마스터도 둘. <code>t_customer</code> 18,543행, <code>t_partner</code> 9,513행. 스키마도 다름(<code>customer_type</code> vs <code>partner_type</code> ·여신·결제조건).	같은 거래처의 여신·미수·납품이력이 한 화면에 모이지 않음.
God Object 한 객체에 여러 실체	hisense <code>t_order</code> 는 28컬럼 중 9개가 거래처 실체(<code>ordering_org</code> , <code>customer_org</code> , <code>customer_phone</code> , <code>customer_fax</code> , <code>customer_address</code> , <code>customer_contact</code> ...). 정작 <code>t_customer</code> 는 0행.	발주기관·수요기관이 문자열로만 존재 → 기관 단위 집계·명부 대조 불가.
Kitchen Sink 의미 없는 범용 슬롯	naturefood <code>mes_item</code> 48컬럼에 <code>item_param1~4</code> , <code>envelop_name1~3</code> / <code>qty</code> / <code>unit</code> , <code>clt_ctgr1~4</code> 같은 이름만 있고 의미는 코드에만 있는 필드가 절반 가까이.	필드 의미가 화면 코드에 흩어져 신규 화면마다 재해석.
Join Key를 관 계로 착각 테이블을 그대로 객체로	<code>t_order_item</code> 의 PK 이름이 <code>item_id</code> 인데, 이것은 주문 라인의 PK이고 <code>t_item.item_id</code> 는 품목 마스터의 PK다. 이름만 같은 두 ID 공간.	hisense는 16개 생성지점 의미통일로 해결했다가 읽기속 12개 파일에서 회귀 장애(2026-08-07). infurs는 같은 문제를 <code>master_item_id</code> 컬럼 추가로 회피 — 같은 문제, 다른 해법.
공통 언어 부재	같은 "품목"이 <code>item_id</code> (infurs·hisense) / <code>it_id</code> · <code>it_no</code> (naturefood), "수주"가 <code>t_order</code> / <code>mes_sale_order</code> , "작업지시"가 <code>t_work_order</code> / <code>mes_production_work</code> .	한 사이트에서 만든 화면·규칙·점검 스크립트를 다른 사이트에 재사용 못 함.

세 MES는 데이터가 부족한 게 아니라, 같은 현실을 가리키는 이름이 세 벌인 상태다. 온톨로지가 해결하는 것은 저장이 아니라 이 이름 문제다.

1.1 이미 온톨로지가 도는 선례

맨땅이 아니다. 케어앤(caren)에서 이미 Apache Jena Fuseki를 도입해 운영 중이다 — `moai-fuseki` 컨테이너(7주 연속 가동) 안에 `/moai` 와 `/caren` 두 데이터셋이 공존하고, `care-domain.ttl` 271 트리플로 질병→특기 추론(`requiresSpecialty` + `broaderSpecialty` 양방향 폐쇄)과 STT 어휘 개인화를 서비스하고 있다. SPARQL 클라이언트는 표

준 라이브러리만 쓴 140행(`ontology.py`)이며, Fuseki가 죽어도 매칭·STT는 기존 경로로 조용히 풀백한다.

이 설계는 그 선례의 **운영 방식**을 그대로 물려받는다. 새 인프라를 세우는 제안이 아니라, 검증된 배치 위에 `tx-mes` 데이터셋 하나를 더하는 제안이다.

§ 02

설계 원칙 7개

이후 모든 설계 판단은 이 7개에서 파생된다. 원칙과 충돌하는 구현은 구현이 틀린 것으로 본다.

P1 시스템이 아니라 현실을 모델링한다

`t_order_item`을 `OrderLine`으로 옮기는 게 목적이 아니다. 현장 사람이 쓰는 말 — 수주, 품목, 작업지시, 로트, 포장, 출하 — 를 먼저 정의하고 테이블을 거기에 붙인다. 테이블이 없으면 객체를 만들지 않고, 테이블이 들어오면 객체는 하나다.

P2 MES DB가 단일 진실원천(SSOT)이다

온톨로지는

읽기 투영(read model)

이다. 트리플스토어에 원본만 있는 데이터는 만들지 않는다. 온톨로지를 지우고 다시 만들어도 업무는 멈추지 않아야 한다.

P3 IRI로 ID 공간을 물리적으로 분리한다

hisense가 치른 `item_id` 회귀는 두 ID 공간이 같은 *이름*을 썼기 때문에 생겼다. IRI는 클래스를 이름에 포함하므로 같은 실수가 구조적으로 불가능하다(§ 4.2).

P4 시계열은 온톨로지에 넣지 않는다

naturefood `mes_gathering_sensor`는 85,610행이고 계속 늘어난다. 온톨로지에는 `SensorChannel` 정의·임계값·설비 연결만 두고, 측정값은 SQL에 남긴다. 판단에 쓰는 것은 "가동/비가동" 같은 **해석된 상태**다.

P5 Action은 온톨로지가 실행하지 않는다

쓰기는 전부 기존 MES 엔드포인트(Command)를 거친다. 이미 검증된 불변식 — 3경로 이중차감 방지, 출고 상한, 포장 내역서=실제출고 — 을 우회하는 쓰기 경로를 새로 만들지 않는다(§ 6).

P6 **자연어는 조건만 파싱하고, 실행은 화이트리스트가 한다**

hisense 품목조회·naturefood 대시보드 NL검색에서 이미 채택한 원칙. LLM이 SPARQL/SQL을 생성해 그대로 실행하는 구조는 채택하지 않는다.

P7 **폴백 경로를 제거하지 않는다**

Fuseki 미기동·타임아웃 시 모든 화면은 온톨로지 없이 기존 SQL 경로로 동작해야 한다. 타임아웃 상한은 caren과 동일하게 1.5s 에서 시작한다.

§ 03

공통 코어 온톨로지

네임스페이스는 `http://mes.telomx.kr/ontology#` (접두어 `tx:`). 세 사이트가 클래스와 관계는 공유하고, 개체는 사이트별 named graph에 들어간다(§ 9.2). 클래스는 14개로 시작한다 — 세 MES에 실제로 테이블이 있고, 실제 질문에 쓰이는 것만 넣었다.

3.1 객체 그래프

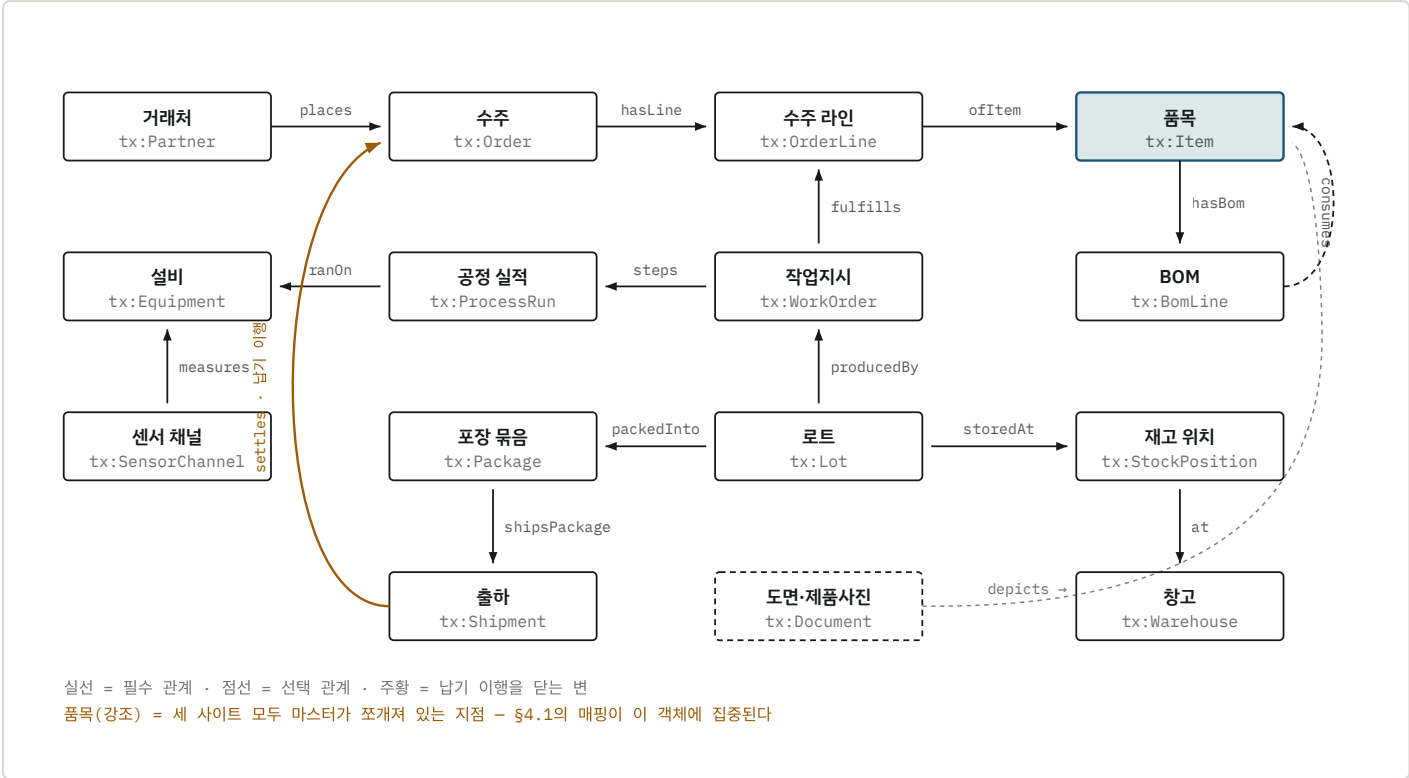


그림 1. 코어 객체 그래프. 오른쪽 위 tx:Item 을 강조한 이유는 이 객체가 세 사이트에서 각각 다르게 쪼개져 있어서다(infurs는 t_item + t_product , hisense는 차종·ALC 속성 인라인, naturefood는 mes_item 48컬럼). 주황색 변 settles 는 "이 수주가 실제로 납품됐는가"를 답하는 경로로, 현재는 수주·포장·배송지시 3경로가 따로 집계돼 누락이 났던 구간이다.

3.2 클래스 정의

클래스	현실의 무엇	핵심 PROPERTY (선별)	추정 개체수
tx:Partner	거래처 — 발주기관·수요기관·고객사·공급사를 한 객체로	bizNo, name, role(발주/수요/공급), creditLimit	28,000+
tx:Order	수주 1건	orderNo, orderDate, deliveryDeadline, status, grandTotal	38
tx:OrderLine	수주의 품목별 한 줄 — 잔량 판단의 기준 단위	lineNo, qty, unitPrice, deadline, identifyNo	66
tx:Item	품목 — 제품·부자재·원자재 공통	itemCode, identifyNo, itemType, unit, safetyStock, leadTimeDay	1,114
tx:BomLine	품목 1단 소요 구성	qty, unit, lossRate	1,270
tx:WorkOrder	작업지시	woNo, targetQty, productionDeadline, status	57

클래스	현실의 무엇	핵심 PROPERTY (선별)	추정 개체수
tx:ProcessRun	공정별 진척 실적	process, startAt, endAt, goodQty, defectQty	120
tx:Lot	로트 — 추적성의 단위	lotNo, qty, pkgType, expiryDate, productionDate	180
tx:Package	포장 묶음 (hisense QR 라벨 단위)	packageNo, pkgType, packedAt, shippedAt	33
tx:StockPosition	창고×품목(×로트) 현재고	qty, updatedAt	212
tx:Warehouse	창고·구역	name, type, sectionName	22
tx:Shipment	출하 — 배송지시·포장출하·판매출고 3경로의 공통 상위	route(order/package/delivery), dispatchDate, deliveredAt, status	42
tx:Equipment	설비	eqCode, name, model, maker	20
tx:SensorChannel	센서 채널 정의 — 측정값은 넣지 않음 (P4)	channel, unit, runThreshold, sourceTable	6

개체수는 세 DB 합계 실측(`information_schema` + `count(*)` , 2026-09-19 기준)이다.

`tx:Partner` 만 유독 큰 것은 infurs `t_customer` 18,543 + `t_partner` 9,513 + naturefood `mes_client` 8,494의 중복 정리 전 수치라서다 — 정리 후 실제 규모는 § 10 Phase 1에서 확정한다.

3.3 관계(ObjectProperty)

관계	도메인 → 레인지	이 관계가 답하는 업무 질문
tx:places	Partner → Order	이 기관이 올해 발주한 수주는?
tx:hasLine	Order → OrderLine	이 수주에 남은 품목별 잔량은?
tx:ofItem	OrderLine · Lot · StockPosition → Item	이 품목이 걸린 수주·재고·로트 전부
tx:hasBom	Item → BomLine	이 제품을 만들려면 무엇이 필요한가?
tx:consumes	BomLine → Item	이 자재가 부족하면 못 만드는 제품은?(역방향 전이)
tx:fulfills	WorkOrder → OrderLine	이 작업지시는 어느 수주 라인을 위한 것인가?
tx:steps	WorkOrder → ProcessRun	이 지시는 어느 공정까지 왔나?
tx:ranOn	ProcessRun → Equipment	이 설비가 멈추면 지연되는 지시는?
tx:measures	SensorChannel → Equipment	이 설비의 가동 판정 근거 채널은?
tx:producedBy	Lot → WorkOrder	이 로트는 언제·어느 지시로 만들어졌나?

관계	도메인 → 레인지	이 관계가 답하는 업무 질문
tx:storedAt	Lot · StockPosition → Warehouse	어느 창고에 얼마 있나?
tx:packedInto	Lot → Package	이 로트는 어느 묶음으로 나갔나?
tx:shipsPackage	Shipment → Package	이 출하에 실린 묶음·로트
tx:settles	Shipment → OrderLine	이 수주는 실제로 다 납품됐나? (3경로 통합)
tx:depicts	Document → Item	이 품목의 도면·제품사진
tx:sameAs	Item → Item (사이트 간)	세 공장이 같은 자재를 쓰고 있나?(Phase 4)

설계 판단

`tx:consumes` 와 `tx:hasBom` 의 **역방향 전이 폐쇄**가 이 온톨로지의 실질적 수익원이다. "이 부자재가 떨어지면 어느 수주의 납기가 위험한가"는 지금 SQL로는 BOM 단계마다 조인을 새로 써야 하지만, SPARQL에서는 `?item (^tx:consumes/^tx:hasBom)* ?product` 한 줄이다. `caren`의 `broaderSpecialty` 양방향 폐쇄와 같은 패턴을 그대로 쓴다.

§ 04

사이트별 매핑과 IRI 규칙

이 표가 설계서의 실질적 산출물이다. 클래스마다 세 사이트의 실제 테이블·컬럼을 명시한다. 빈칸은 그 사이트에 해당 실체가 없다는 뜻이고, 표에 담긴 불일치가 그대로 Phase 1의 작업 목록이 된다.

4.1 객체 → 테이블 매핑

클래스	INFURS_DB	HISENSE_DB	NATUREFOOD	주의
Partner	t_customer + t_partner	t_partner (2행)	mes_client	infurs 이중 마스터, hisense는 사실상 비어 있음
Order	t_order	t_order	mes_sale_order	hisense는 거래처가 이 테이블에 인라인
OrderLine	t_order_item	t_order_item	mes_sale_item	PK가 item_id — 품목 PK와 동명(§ 4.2)
Item	t_item + t_product	t_item	mes_item	infurs는 memo 문자열로만 조인 가능

클래스	INFURS_DB	HISENSE_DB	NATUREFOOD	주의
BomLine	t_bom / t_bom_detail	t_bom / t_bom_detail	mes_production_bom	naturefood는 BOM이 작업(work_idx)에 매 달림
WorkOrder	t_work_order (master_item_id 有)	t_work_order (컬럼 無)	mes_production_request / _work	같은 문제를 다르게 해 결한 지점
ProcessRun	t_production / t_process_step	t_production	mes_production_work / _time	naturefood는 공정 시 간이 별 테이블
Lot	—	t_lot (180행)	mes_warehouse_stock (유 통기한)	infurs는 로트 개념 없 음 — 객체를 만들지 않 는다(P1)
Package	—	t_package / t_package_item	mes_ship_item	hisense만 QR 라벨 단 위 존재
StockPosition	t_stock	t_stock / t_stock_transfer	mes_warehouse_stock	naturefood는 구역 (section_name)까 지
Warehouse	t_warehouse (3)	t_warehouse (2)	mes_warehouse_in_out	
Shipment	t_delivery / t_delivery_item	t_delivery + t_package + 판매출 고	mes_ship_record / mes_shipping_order	hisense 3경로 통합이 핵심
Equipment	t_facility (0행)	t_facility (0행)	mes_equipment (11행)	설비 마스터는 naturefood만 실데이 터
SensorChannel	—	—	mes_gathering_sensor a1~a6	채널 정의만, 측정값 85,610행은 SQL 잔류 (P4)
Document	t_drawing / t_product_picture	t_item_file	attached_files	hisense는 전용 테이블 없이 공용 파일표 재사 용

4.2 IRI 규칙 — ID 이중공간을 구조적으로 없앤다

2026-08-07 hisense 장애의 원인은 item_id 라는 이름이 두 가지를 가리켰다는 것이다. 16개 생성 지점의 의미를 통일했더니 읽기축 12개 파일이 옛 의미를 그대로 읽어 회귀했다. IRI는 클래스와 사이트를 이름 안에 포함하므로, 두 ID가 우연히 같은 문자열이 되는 일이 구조적으로 불가능하다.

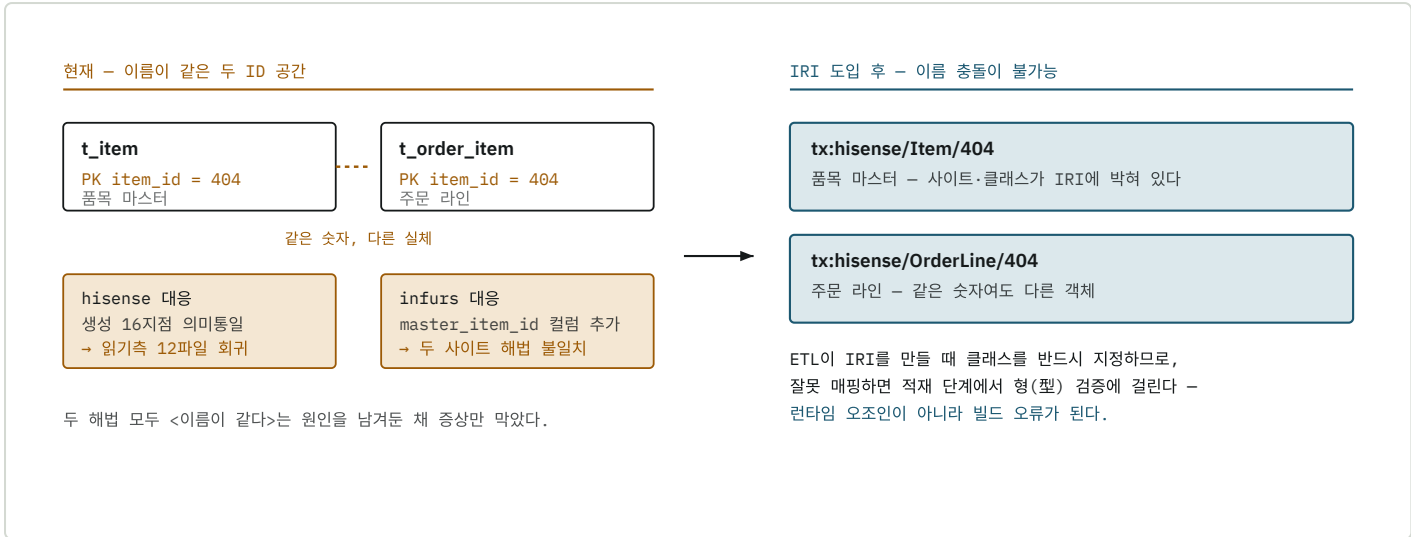


그림 2. 같은 결함에 대한 두 사이트의 대응과, IRI가 그 결함을 없애는 방식. 핵심은 "온톨로지가 더 똑똑해서"가 아니라 **오류 발생 시점이 런타임에서 적재 시점으로 옮겨진다는 점**이다.

IRI 생성 규칙

`http://mes.telomx.kr/id/{site}/{Class}/{naturalKey}`

- `{site}` - infurs | hisense | naturefood. 사이트를 IRI에 넣어 named graph와 이중 방어.
- `{Class}` - § 3.2 클래스명 그대로. 이 조각이 ID 공간을 분리한다.
- `{naturalKey}` - PK가 아니라 업무 키를 우선한다. Order 는 order_no, Item 은 item_code, Lot 은 lot_no. PK는 업무 키가 없을 때만 쓰고 그 경우 pk- 접두어를 붙인다 (OrderLine/pk-23).
- 업무 키를 쓰는 이유: 전체 재적재(full reload) 후에도 IRI가 안 바뀌어 외부에서 인용한 링크가 유지된다.

§ 05

Logic — 판단 규칙 등록부

판단 규칙은 이미 전부 존재한다. 문제는 그것이 *PHP 컨트롤러 안에 흩어져* 있어서 같은 규칙이 사이트마다 다르게 구현되고, 어느 화면이 어느 규칙을 쓰는지 아무도 모른다는 것이다. Logic 계층의 목적은 규칙을 새로 만드는 게 아니라 등록하는 것이다 - 온톨로지에는 규칙의 식별자·입력 객체·임계값·구현 위치를 두고, 계산 자체는 기존 코드가 한다.

규칙	입력 객체	현재 구현 위치	등록 형태
안전재고 미달 판정	StockPosition, Item	Inventory.php · t_item.safety_stock / reorder_point	임계값을 tx:Item Property로 승격
BOM 소요량 전개	Item, BomLine, WorkOrder	Bom.php, WorkOrder.php	전이 폐쇄 SPARQL + 수량은 SQL
수주 잔량 / 배송대상 판정	OrderLine, Shipment	hisense 3경로 합산(Dashboard.php, Delivery.php)	tx:settles 경유 단일 질의로 대체
납기 위험	Order, OrderLine, WorkOrder	사이트별 산재	Function 등록 + 위험도 산식 단일화
설비 가동 판정	SensorChannel, Equipment	naturefood Monitor.php 임계값	tx:runThreshold Property (SSOT)
포장 속도 / 리드타임 KPI	ProcessRun, Package	naturefood 공정모니터링 — 640 스틱/분, 포장구간=실적창	KPI 정의를 온톨로지에, 값은 SQL 집계
KPI 목표·실적	전 객체	hisense t_kpi_config (6행) / t_kpi	KPI ↔ 객체 연결을 Link로 명시
품목 자동분류	Item	hisense 조달청 XLS 파싱 item_type 자동분류	분류 규칙을 클래스 계층으로 표현

경계

규칙을 온톨로지로 옮기지 않는다. 옮기면 SSOT가 둘이 되고(P2 위반), 기존 화면이 Fuseki 가동에 의존하게 된다(P7 위반). 온톨로지는 "이 규칙이 존재하고, 임계값은 이것이고, 구현은 여기 있다"를 아는 목록이다. 단, 임계값 숫자는 예외로 온톨로지를 SSOT로 삼는다 — 지금 세 사이트에 흩어진 임계값이 서로 다른 게 실제 문제이기 때문이다.

Action — 실행은 MES가 한다

블로그가 강조한 "Action Loop를 닫는다"는 부분은 우리 환경에서 가장 조심해야 하는 대목이다. 팔란티어는 Foundry가 원천이라 온톨로지가 직접 write-back한다. 우리는 반대로 **MES DB가 원천**이고, 그 DB에는 이미 피로 배운 불변식이 걸려 있다.

이미 존재하는 불변식 — 우회하면 안 되는 것들

- 3경로 이중차감 방지 — 판매출고가 sales_order / package / delivery 세 경로로 발생하며, 같은 수량이 두 번 차감되면 안 된다.

- **출고 상한** — 배정 로트 분기가 상한을 적용하지 않아 초과출고가 났던 이력. 상한은 품목별 잔량 기준.
- **포장 내역서 = 실제 출고** — 이 등식은 확정된 불변식이다.
- **작업지시 생성은 공용 헬퍼만** — `resolve_master_item_id()` / `link_work_order_to_plan()`. 16지점 일원화 결과를 되돌리는 경로를 만들지 않는다.
- **마이그레이션 실패 시 코드까지 롤백되는 배포 스크립트가 있다** — 스키마 변경은 최소화한다.

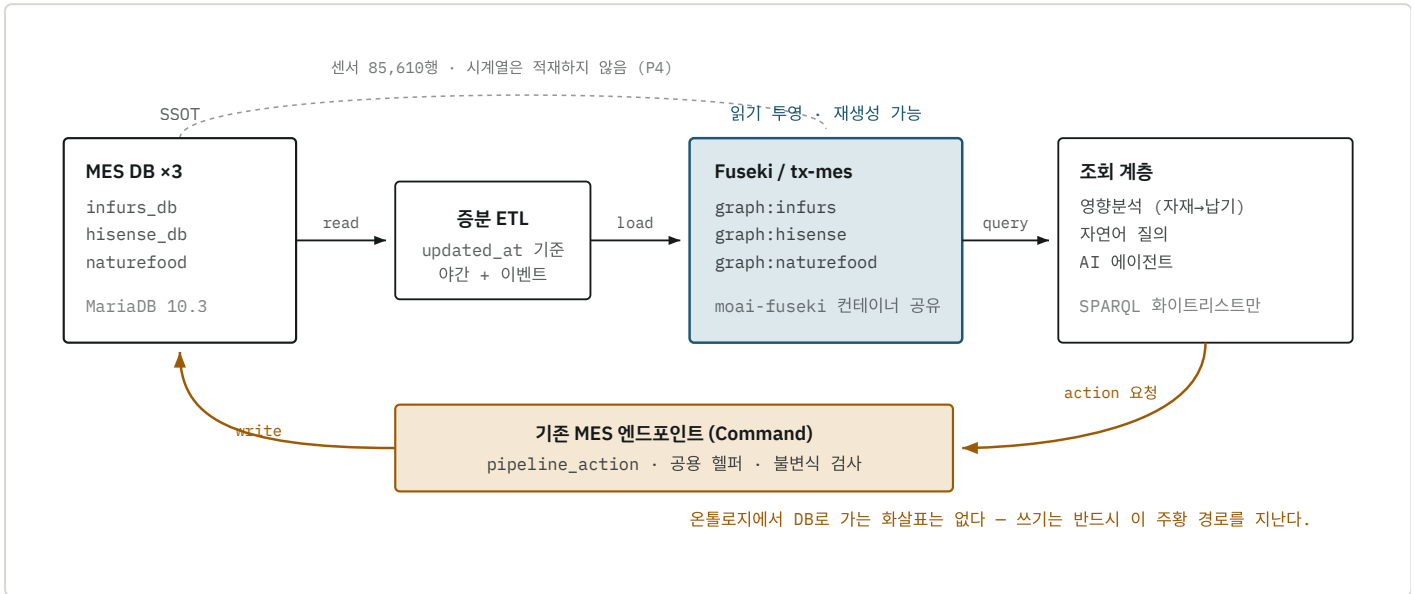


그림 3. 배치와 Action 루프. 온톨로지는 조회 계층에만 답하고, 상태를 바꾸는 요청은 전부 기존 MES 엔드포인트를 거쳐 불변식 검사를 받는다. 이 구조면 Fuseki가 죽어도 업무가 멈추지 않고(P7), 온톨로지를 전체 재적재해도 데이터가 유실되지 않는다(P2).

Action 정의 형식

온톨로지에 등록하는 Action은 "무엇을 할 수 있는가"의 선언이며, 실행 주체는 항상 MES다. 각 Action은 ① 대상 클래스 ② 호출할 엔드포인트 ③ 필요 권한 ④ 검증할 불변식 네 가지를 갖는다. 1차 등록 대상은 이미 화면이 있는 것들로 한정한다 — 작업지시 발행, 공정 진척 기록, 포장 묶음 생성, 출하 확정, 재고 이동, 배송완료 처리.

§ 07

Security — 사람과 AI를 같은 체계로

AI가 조회·실행을 하기 시작하면 권한은 부가기능이 아니라 구조가 된다. 다행히 우리에게 이미 권한 테이블이 있다 — `t_user` / `t_permission` / `t_menu` (infurs·hisense), `mes_menu_users` 430행(naturefood). Action의 권한은 새로 정의하지 않고 이 메뉴 권한에 매핑한다.

- **Fuseki는 localhost 바인딩을 유지한다.** `moai-fuseki`는 host network + `--localhost`로 띄워 있다. 외부 노출 금지 — 웹에서 SPARQL 엔드포인트를 직접 부르는 구조를 만들지 않는다.

- **질의는 서버측 화이트리스트만.** 파라미터만 바인딩하는 고정 SPARQL 템플릿. 자연어는 조건 파싱에만 쓴다(P6).
- **행 단위 제한은 named graph로.** 사용자가 속한 사이트의 graph만 질의 대상에 포함한다. 사이트 간 조회는 Phase 4에서 별도 권한으로.
- **AI 에이전트의 도구 범위 = 그 계정의 메뉴 권한.** 사람이 못 하는 일을 에이전트가 할 수 있는 경로를 만들지 않는다.
- **선결 조치.** login.php 에 admin 비밀번호가 평문으로 박혀 있는 문제가 infurs·hisense 양쪽에 남아 있다. 온톨로지 조회 계층을 붙이기 전에 이것부터 정리해야 한다 — 권한 체계 위에 AI를 올리는 설계에서 최초 관문이 뚫려 있으면 나머지 설계가 무의미하다.

§ 08

Decision Lineage

"데이터가 어디서 왔나"(Data Lineage)보다 넓은 "누가 무엇을 보고 어떤 판단을 했고 그 결과가 어땠나"를 기록하자는 개념이다. 우리는 이미 조각을 갖고 있다 — 다만 서로 연결되지 않은 로그들이다.

기존 로그	담고 있는 것	LINEAGE로 승격시 추가할 것
t_audit_log	user_id, action, target, ref_id, summary, ip	대상을 IRI로 기록 — 테이블명+PK 대신
t_ai_parse_log	OCR 텍스트, AI 결과, 정규식 결과, parse_method	어느 Logic·모델 버전이 쓰였는지
t_ocr_correction	사람이 고친 값	판단 주체(사람/AI)와 수정 근거 — 재학습 입력
ai_rule_history	규칙 변경 이력 (현재 0행)	임계값 변경 → 영향받은 판단 연결
t_document_log	문서 발행 이력 17행	어느 Order·Shipment IRI에 귀속되는지

최소 변경 원칙

새 로그 테이블을 만들지 않는다. t_audit_log.target 에 들어가는 문자열을 IRI 형식으로 바꾸는 것만으로 다섯 로그가 같은 객체를 가리키게 된다. MariaDB 10.3에서 AFTER · INSTANT 컬럼 추가가 지원되지 않는 제약도 이 방식이면 피할 수 있다.

§ 09

배치 아키텍처와 동기화

9.1 서버 제약이 설계를 결정한다

이 서버는 RAM 3,665MB에 가용 1,320MB다. 새벽 백업의 git bundle이 반복 OOM kill을 당한 이력이 있고, Claude 세션에는 cgroup 캡이 걸려 있다. 따라서 **새 컨테이너·새 JVM을 띄우지 않는다**.
caren과 동일하게 이미 7주 가동 중인 `moai-fuseki`에 `tx-mes` TDB2 데이터셋 하나만 추가한다.

9.2 트리플 규모 추정

NAMED GRAPH	업무 객체 행수 (실측)	객체당 트리플	추정 트리플
graph:infurs	30,659	8~12	245K ~ 368K
graph:hisense	2,417	10~14	24K ~ 34K
graph:naturefood	9,774	8~12	78K ~ 117K
graph:schema	클래스 14 · 관계 16	—	~400
합계 (세션·로그·시계열 제외)			35만 ~ 52만

TDB2에서 50만 트리플은 수백 MB 힙으로 충분하며 caren(271 트리플)과 같은 인스턴스를 공유해도 문제없는 규모다. 단, `graph:infurs`의 80%가 거래처 중복이므로 Phase 1의 Partner 정리로 절반 이하로 줄 가능성이 크다. **naturefood** 센서 85,610행을 넣으면 규모가 이 추정과 무의미해진다 — P4를 지키는 이유다.

9.3 동기화 방식

- 1차: 야간 전체 재적재. 50만 트리플이면 전체 재적재가 수분 내다. 증분 로직의 버그 위험보다 단순함이 낫다. 기존 백업 크론 시간(OOM 이력)과 겹치지 않게 배치한다.
- 2차: 이벤트 증분. 수주 등록·작업지시 발행·출하 확정 시점에 해당 객체만 갱신. `updated_at` 기준.
- 실패 처리. 적재 실패 시 이전 그래프를 유지한다(부분 적재 금지). 적재 전 `riot --validate`로 TTL 문법 검증 — caren에서 쓰던 절차 그대로.
- 불변식 점검. infurs의 `check_invariants.php` 패턴을 확장해, 적재 후 객체 수·필수 Link 누락을 상시 점검한다.
- 외부 의존 없음. 이 서버는 npm·jsdelivr·Docker Hub·Cloudflare 대역이 차단돼 있다. ETL은 서버에 이미 있는 PHP CLI 또는 표준 라이브러리 Python으로만 만든다(caren `ontology.py`가 `urllib`만 쓴 이유).

단계별 로드맵

각 단계는 그 단계만으로 눈에 보이는 성과가 있어야 한다. "온톨로지를 다 만든 다음에 효과가 난다"는 계획은 중간에 멈추면 전부 손실이므로 채택하지 않는다.

Phase 0

합의

용어 합의와 스키마 확정

- § 3.2 클래스 14개· § 3.3 관계 16개를 현장 용어로 검토 — 세 사이트 담당자가 같은 말을 쓰는지 확인
- `tx:` 네임스페이스·IRI 규칙(§ 4.2) 확정, `mes-core.ttl` 작성 후 `riot --validate`
- Fuseki에 `tx-mes` 데이터셋 생성 + `graph:schema` 적재

산출물 `mes-core.ttl` · 용어 대조표 · 빈 데이터셋 · 검증 문법 검증 통과, 트리플 ~400

Phase 1

적재

한 사이트 전체 적재 — hisense 먼저

- hisense를 1번으로 두는 이유: 객체 2,417행으로 가장 작고, Lot·Package·3경로 출하가 *유일하게 다 있는* 사이트라 모델 검증이 제대로 된다
- ETL 작성(PHP CLI) → `graph:hisense` 적재 → 객체 수·필수 Link 누락 점검
- 부수 성과: 적재가 실패하는 행이 곧 데이터 결함 목록이 된다(고아 명세서·미연결 도면 등)

산출물 ETL 스크립트 · 적재 리포트 · 결함 목록 · 검증 2.4만~3.4만 트리플, 필수 Link 누락 0

Phase 2

첫 수익

자재 영향분석 화면 — 온톨로지가 아니면 못 만드는 것

- 질문: "이 부자재가 떨어지면 어느 수주의 납기가 위험한가?" — `^tx:consumes/^tx:hasBom` 전이 폐쇄
- 현재 SQL로는 BOM 단계마다 조인을 새로 써야 해서 화면이 없다. SPARQL 한 질의로 대체
- `tx:settles` 로 3경로 출하를 합산해 수주 잔량을 단일 질의로 — 3차 스왑에서 손으로 맞춘 로직의 계산에도 쓴다
- Fuseki 미기동 시 "영향분석 일시 사용 불가" 표시 후 기존 화면 정상 동작(P7)

산출물 영향분석 화면 1개 · 화이트리스트 SPARQL 템플릿 · 검증 기존 수주잔량 집계와 수치 일치

Phase 3

확산

infurs·naturefood 적재와 Partner 정리

- infurs: `t_item` + `t_product` 를 하나의 `tx:Item` 으로 — `memo` 문자열 조인을 IRI 동일성으로 대체. 미연결 312품목이 여기서 드러난다
- infurs: `t_customer` (18,543)+ `t_partner` (9,513) 중복 정리 → `tx:Partner` 실제 규모 확정
- naturefood: `mes_item` 48컬럼 중 실제 쓰이는 Property만 선별(Kitchen Sink 해소), `SensorChannel` 6채널 정의·임계값 등록
- hisense `t_order` 의 인라인 거래처 9컬럼 → `tx:Partner` 객체로 승격(DB는 그대로, 온톨로지에서 분리)

산출물 3그래프 적재 완료 · 중복 정리 리포트 · 검증 35만~52만 트리플, 사이트별 불변식 점검 통과

Phase 4

운영

Action 등록 · Lineage · 자연어 질의

- Action 6종 등록(§ 6) — 대상 클래스·엔드포인트·권한·불변식 4요소
- `t_audit_log.target` 을 IRI로 전환 → 5개 로그가 같은 객체를 가리킴(§ 8)
- 기존 NL검색(hisense 품목조회·naturefood 대시보드)의 조건 파싱 결과를 SPARQL 화이트리스트로 라우팅
- `tx:sameAs` 로 사이트 간 동일 자재 연결 — 세 공장 공통 구매·재고 재배분 검토의 전제

산출물 Action 등록부 · Lineage 전환 · NL 질의 연결 · 검증 권한 밖 Action 차단 확인, 감사 로그로 판단 추적 가능

전제 조건

Phase 4의 자연어 질의는 LLM 크레딧을 쓴다. hisense 자연어검색·일반수주서 OCR이 이미 **크레딧 부족으로 라이브 미검증** 상태다. Phase 4 착수 전에 크레딧 확보 여부를 먼저 확인해야 하며, 확보 안 되면 Phase 4에서 자연어 항목만 분리해 뒤로 미룬다 — Action 등록과 Lineage는 크레딧과 무관하다.

금지 목록과 성공 판정

11.1 하지 않는 것

- 테이블을 그대로 객체로 옮기지 않는다. `t_order_item` 한 행에 품목명·규격·단가가 복사돼 있어도, 객체는 `OrderLine` 과 `Item` 둘로 나눈다.
- 사이트별로 `Item` 클래스를 따로 만들지 않는다. 그것이 System Silos의 재생산이다. 차이는 Property로, 클래스로는 만들지 않는다.
- 쓰이지 않는 컬럼을 Property로 올리지 않는다. naturefood `item_param1~4` 처럼 의미가 코드에만 있는 필드는 의미를 확인한 것만 이름 붙여 올린다.
- 온톨로지에만 있는 데이터를 만들지 않는다. 예외는 임계값(\$ 5)뿐이며, 그것도 DB에 반영 경로를 함께 만든다.
- 기존 화면을 Fuseki 의존으로 바꾸지 않는다. 온톨로지는 새 화면을 추가할 뿐이다.
- LLM이 생성한 질의를 그대로 실행하지 않는다.
- 새 Fuseki 인스턴스·새 컨테이너를 띄우지 않는다. RAM 제약(\$ 9.1).

11.2 성공 판정

다음 네 가지가 되면 이 설계는 성공한 것으로 본다. 되지 않으면 트리플 수와 무관하게 실패다.

판정 항목	판정 기준
영향 추적	"이 자재가 없으면 위험한 수주"를 한 질의로 답한다 — 지금은 답하는 화면이 없다
ID 사고 재발 방지	품목 ID와 주문라인 ID를 혼용한 코드가 적재 단계에서 걸린다 — 지금은 런타임 회귀로 발견된다
재사용	한 사이트에서 만든 질의·점검이 graph 이름만 바꿔 다른 사이트에 돈다
무해성	Fuseki를 정지시켜도 세 MES의 기존 화면이 전부 정상 동작한다

§ 12

미결 사항

설계를 확정하려면 판단이 필요한 항목들이다. 전부 사용자 결정 사항이다.

#	미결 사항	결정이 필요한 이유
D1	Phase 1 대상이 hisense가 맞는지	규모·모델 완결성은 hisense가 유리하지만, 사업계획서 산출물 기한이 걸린 사이트가 따로 있으면 순서가 바뀐다
D2	infurs 거래처 이중 마스터를 DB에서도 통합할 지	온톨로지에서만 합치면 화면은 그대로 둘이다. DB 통합은 별건 작업이며 영향 범위가 크다
D3	<code>t_item</code> + <code>t_product</code> 통합의 최종 방향	온톨로지 동일성으로 덮을지, infurs <code>master_item_id</code> 처럼 브리지 컬럼을 추가할 지
D4	세 사이트 공통 자재 연결(<code>tx:sameAs</code>)을 어디 까지 볼지	공장 간 재고 재배분은 영업·구매 정책 문제여서 기술 판단만으로는 못 정한다
D5	임계값 SSOT를 온톨로지로 옮기는 범위	안전재고·가동판정·KPI 목표 중 어디까지를 온톨로지가 소유할지
D6	Phase 4 자연어 항목의 LLM 크레딧	미확보 시 해당 항목만 분리 연기 필요(§ 10 전제 조건)
D7	<code>login.php</code> 평문 admin 비밀번호 선결 조치 일정	권한 위에 AI를 올리는 설계의 전제(§ 7). 코드 수정과 비밀번호 변경 둘 다 필요